

EXP52 / EXP53 · GS_FLOATERLAB · 정정판

VIGS-SLAM 실시간성 타임라인

직렬 체인과 GS Mapping, 얼마나 겹치고 어디가 남는가

1253 시퀀스 · 65.1초 실녹화 · imu_cpp+TensorRT 가속 적용 · 타이밍 버그 수정 후 재검증 · 2026-07-20

SUMMARY

세 가지 질문에 대한 답 (정정판)

RTX 5090이면 실시간이 될까?

부분적으로만. VIGS 저자 공식 벤치마크(RTX 5090)에서도 tracking만은 39.83fps(여유)지만 tracking+mapping은 12.02fps로 여전히 목표(30fps) 미달 — GPU 업그레이드는 mapping 병목을 혼자 해결 못 함. (구 버전에서 언급한 "21.1초 미계측 Python 오버헤드"는 아래 정정 참조 — 실체가 밝혀지고 해소됨.)

직렬 체인이 gs_mapping보다 작은가?

거의 같다 — 58.6초 vs 85.8초(둘 다 경합 없는 상태 기준). 어느 한쪽만 0으로 줄여도 나머지 하나가 이미 예산(65.1초)을 넘는다(gs_mapping 단독으로도 초과). 두 축을 동시에 줄여야만 실시간에 도달한다.

⚠ 정정: "27.0초 오버헤드"는 실은 버그였다

실제 계측(pbar/save_trajectory 추가)으로 확인해보니 Python 루프 자체는 0.14초뿐 — 대신 demo.py의 리더 프로세스가 time.sleep(20) 후 종료하는데 reader.join()이 타이밍 마커보다 먼저 실행되던 위치 버그를 발견. 수정 후 모든 "온라인 루프 총합"이 ~20초씩 낮아짐 (180.10→150.56초, 133.04→98.94초) — 아래 슬라이드부터 전부 정정된 수치.

⚠ CORRECTION

타이밍 버그 발견·수정 — 모든 수치 재검증

원인

demo.py 리더 서브프로세스(mono_stream)가 마지막 프레임 큐잉 후 `time.sleep(20)` → 종료. `reader.join()`이 `TRACK_LOOP_DONE_EPOCH` 마커 출력보다 먼저 실행되어 이 인위적 20초가 모든 "온라인 루프 총합"에 섞여 있었음.

영향 없음

`motion_filter/frontend/PGBA/gs_mapping` 등 개별 구성요소 수치, fps 스윙(ORB vs VIGS), evo 궤적 정확도 비교 — 전부 다른 방식(`vigs_track_total` 태그 직접 합산)으로 측정해 이 버그와 무관.

수정

타이밍 마커 출력을 `reader.join()`보다 앞으로 이동(demo.py). 리더 프로세스 정리는 순수 `teardown`이지 측정 대상 워크로드가 아님.

180.10→150.56s

순차: 기존→정정

133.04→98.94s

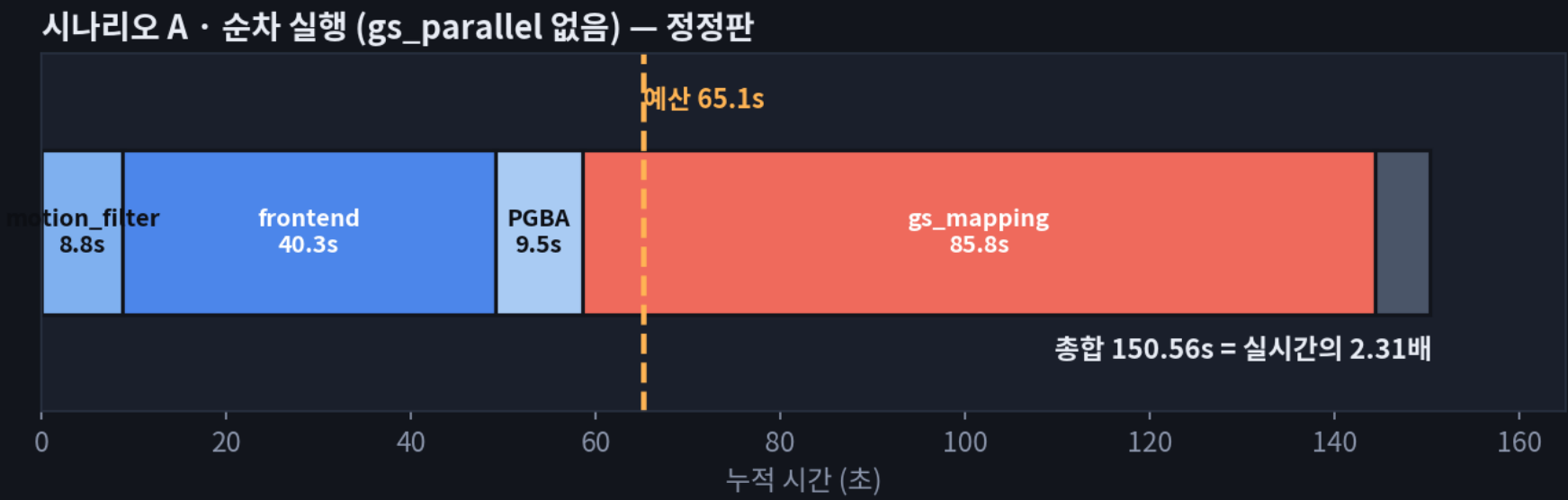
병렬: 기존→정정

21.1s→0.2s

미계측 잔여

실시간 대비 배수도 재계산: 순차 2.77배→2.31배, `gs_parallel` 2.04배→1.52배 — 이전 생각보다 실시간 격차가 훨씬 작다. 결론 방향은 안 바뀜(①`gs_mapping` 연산량 감소 최우선 ②`frontend`는 경합 대응이 핵심 ③오버헤드는 이제 해결됨), `exp53`이 메꿔야 할 목표만 작아짐.

순차 실행 — gs_parallel 없이 돌리면 (정정판)



150.56s
총합

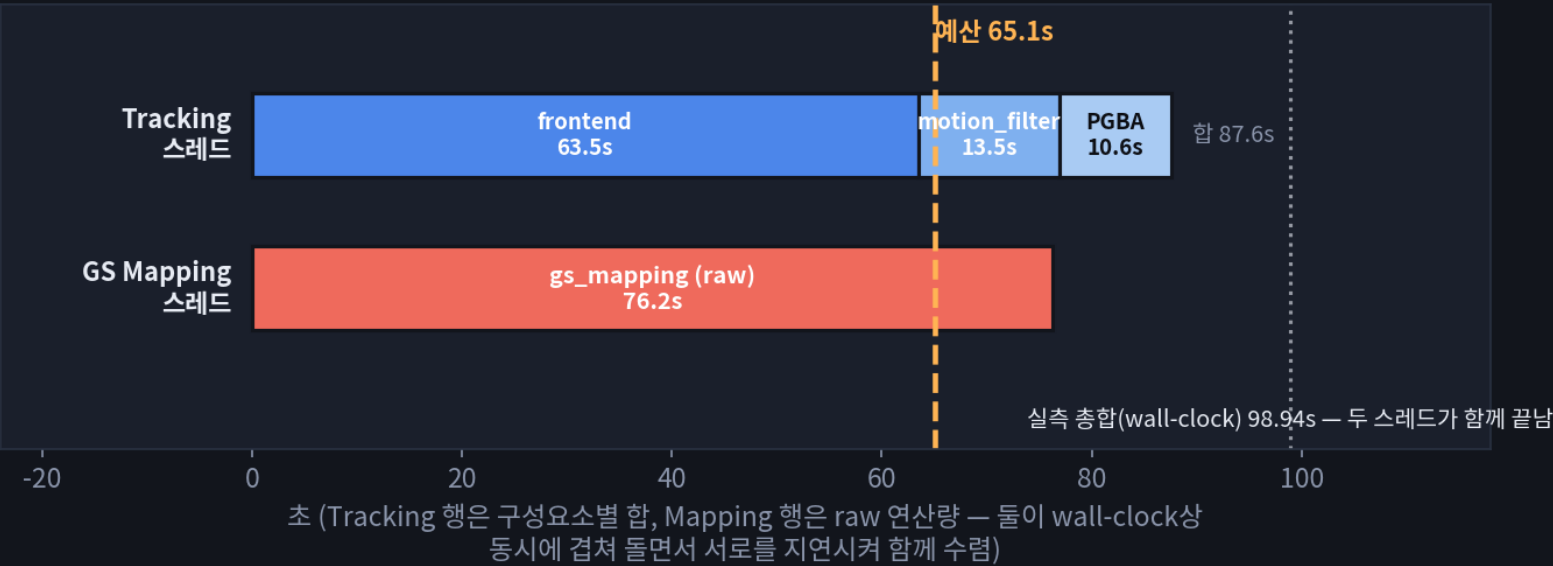
2.31×
실시간 배수

57.0%
gs_mapping 비중

한 프레임당 motion_filter→frontend→PGBA→gs_mapping이 전부 같은 스레드에서 순서대로 실행된다고 가정했을 때, 전체 시퀀스에 걸쳐 각 단계가 누적인 시간의 구성. 오버헤드는 6.0초로 축소(리더 프로세스 타이밍 버그 수정 후).

gs_parallel — 두 스레드가 GPU를 나눠 쓰면 (정정판)

시나리오 B · gs_parallel — Tracking 스레드 내부 구성 — 정정판



98.94s
총합(실측)

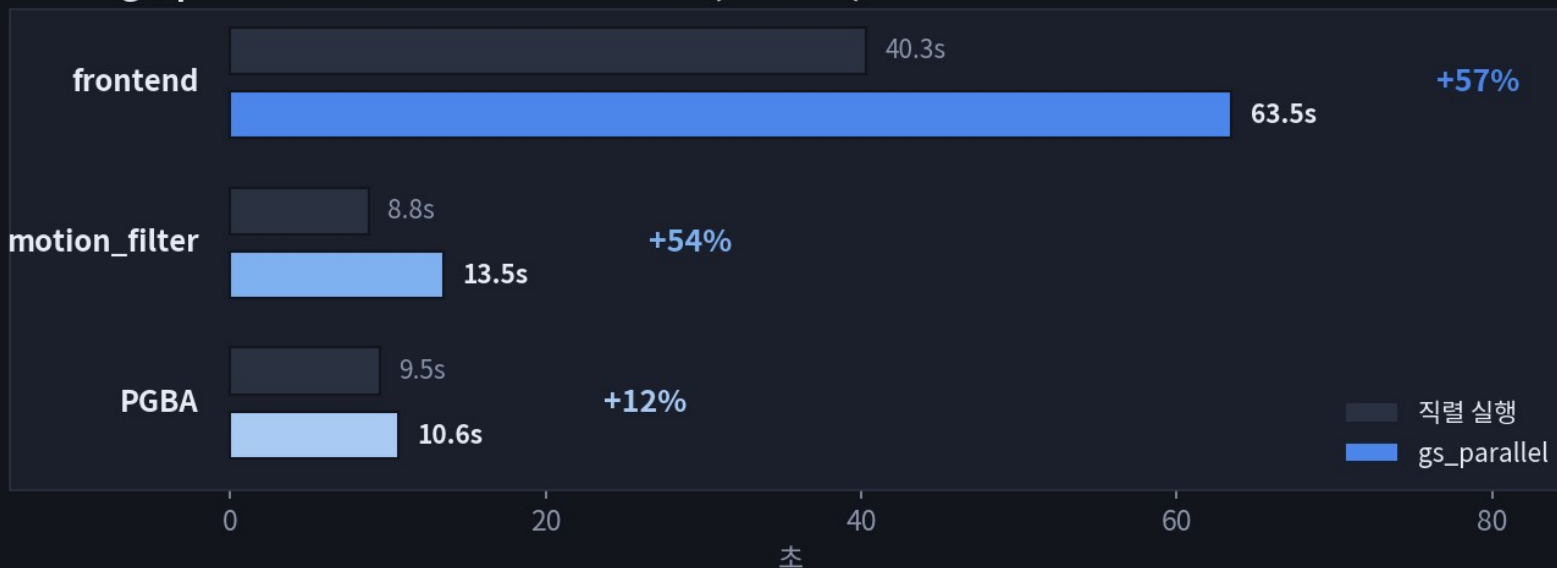
1.52×
실시간 배수

-34.3%
순차 대비

두 스레드 다 98.94초에 함께 끝난다 — 한쪽이 먼저 끝나고 기다리는 구조가 아니라, 서로가 서로를 GPU 경합으로 늦추며 같은 시점에 수렴한다.

왜 frontend가 병렬일 때 오히려 느려지나 (정정판)

직렬 vs gs_parallel — 구성요소별 자체 비용 변화 (경합 영향) — 정정판

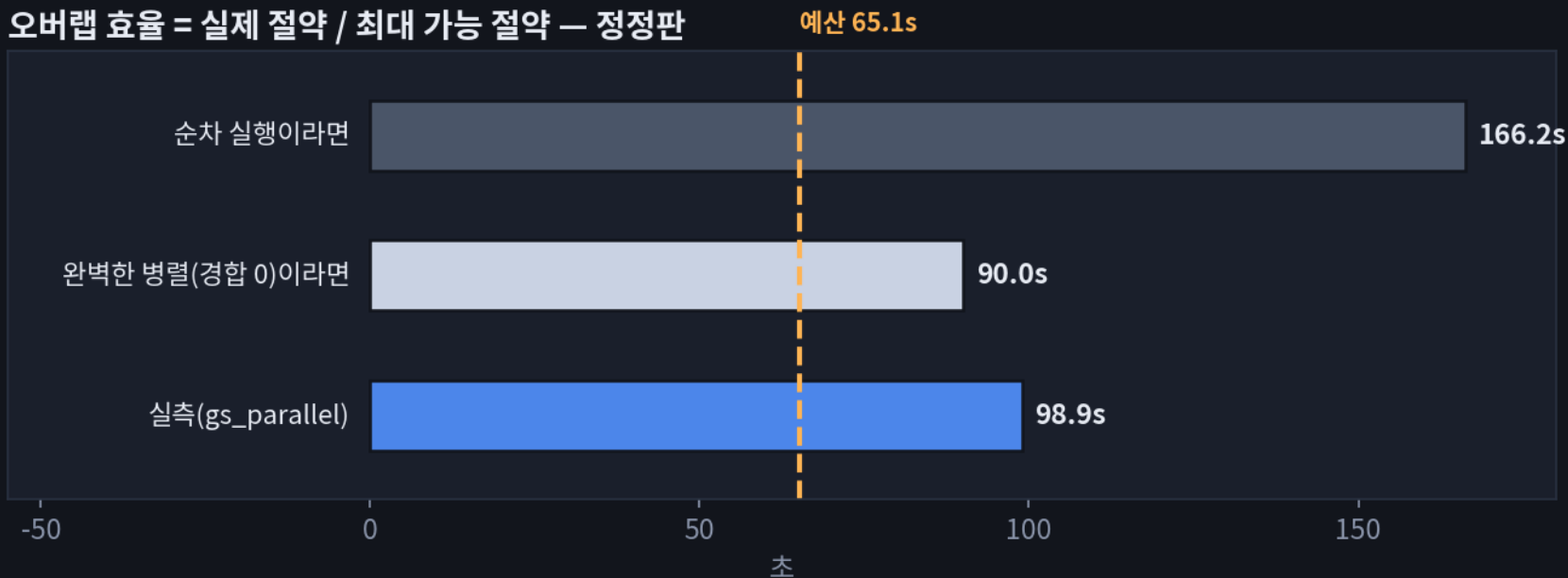


GPU 경합은 매핑 쪽에만 일어나는 게 아니라 양방향이다 — bundle_adjust(BA solve)와 update_op_forward(GRU)는 frontend에서 가장 무거운 GPU 커널인데, rasterize/backward처럼 gs_mapping이 던지는 무거운 커널과 같은 SM·메모리 대역폭을 놓고 경쟁한다.

반면 PGBA(pgba_run)는 sparse pose graph 최적화라 상대적으로 가벼워 거의 안 느려짐(+12%) — mapping 자체도 거의 그대로. 가설(미검증, nsys 트레이스 필요): mapping이 던지는 크고 지속시간 긴 커널 뒤에서 frontend의 작고 빈번한 커널들이 줄서서 기다린다.

오버랩 효율 — 실제로 얼마나 겹치는가 (정정판)

오버랩 효율 = 실제 절약 / 최대 가능 절약 — 정정판



실제 절약 = 166.21 - 98.94 = 67.27s
최대 가능 절약 = 166.21 - 89.99 = 76.22s
오버랩 효율 = 67.27 / 76.22 = 88.3% (나머지 11.7% = GPU 경합으로 못 숨긴 시간)

88.3%

오버랩 효율

11.7%

GPU 경합으로 손실

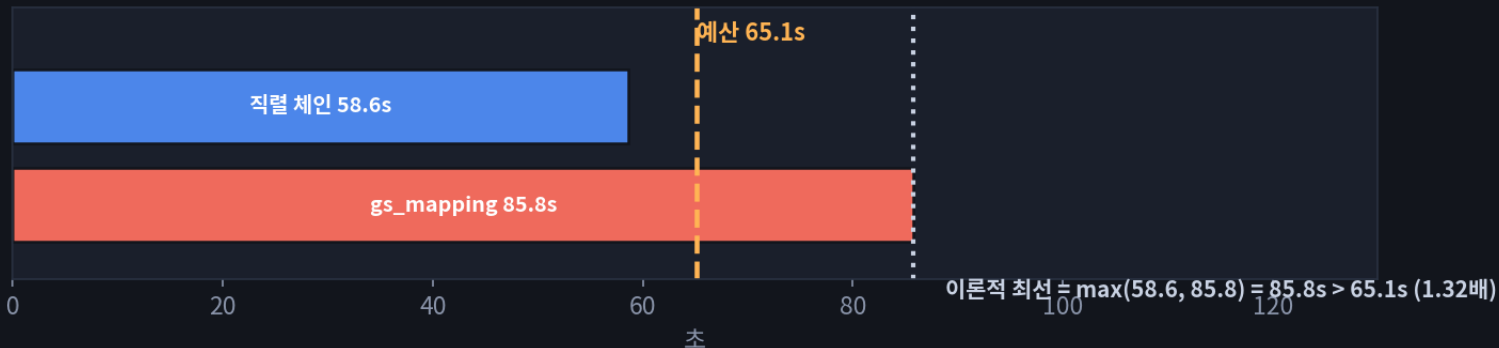
89.99s

Tracking 실측(같은 런)

※ 89.99s는 "경합이 없었을 때의 tracking 비용"이 아니라 이 gs_parallel 런에서 실측된 값 — 그 자체로 이미 위 슬라이드의 경합을 포함하고 있다.

완벽한 병렬화를 가정해도 안 되는 이유 (정정판)

완벽한 병렬화를 가정해도 도달 불가능한 예산 — 정정판



병렬화(exp52)만으로는 구조적으로 도달 불가 — 직렬 체인과 gs_mapping을 각각 65.1초 아래로 줄여야 한다(격차는 기존 추정정보보다 작아짐: 1.39배 → 1.32배)

EXP53 · 최우선

Frontend 반복 축소(iters1/iters2) — bundle_adjust+update_op_forward가 frontend의 대부분. 직렬 체인을 줄이는 가장 직접적인 레버.

EXP53

keyframe 밀도 억제(motion_filter.thresh) — keyframe 발생이 fps와 무관하다는 게 확인됐으니, 밀도 자체를 낮추는 게 fps를 낮추는 것보다 유효.

EXP52 · 해소됨

27.0초 오버헤드 → 리더 프로세스 타이밍 버그로 확인·수정됨(21.1초 미계측 잔여가 0.2초로 해소). 더 이상 미스터리 아님.

EXP53 · 신규

축A~D를 gs_parallel 컨 상태/끈 상태 둘 다에서 측정 — frontend가 경합 시 +57% 느려짐이 확인됐으니, 순차 측정만으론 병렬 실전 효과를 과대추정할 수 있음.

EXP52 · 계속

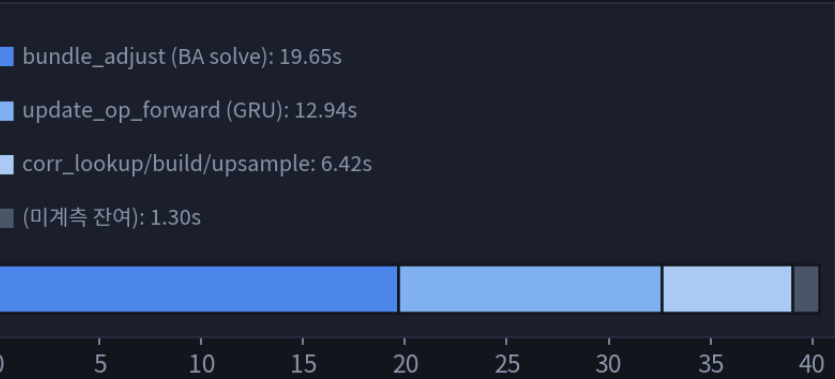
gs_mapping 자체 연산량 감소 — iteration 수·해상도·densify 빈도. RTX 5090에서도 저자 자신이 12.02fps(미달)였던 부분이라 GPU 업그레이드만으론 부족.

COMPONENT BREAKDOWN

구성요소별 세부 분해 (정정판)

구성요소별 세부 분해 — 순차 실행 런 기준, 정정판 (1253 시퀀스 누적 시간)

Frontend Tracking · 40.3s



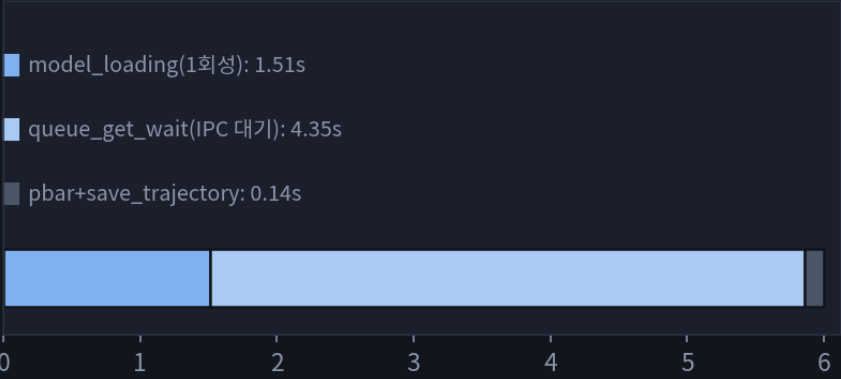
GS Mapping · 85.8s (순차 실행)



motion_filter · 8.8s



track() 바깥 오버헤드 · 6.0s (해소됨)



※ gs_parallel 하에서는 frontend(40.3→63.5s) · motion_filter(8.8→13.5s)가 GPU 경합으로 더 커짐 — 뒤 슬라이드 참조

GS Mapping 루프 최대 세분화

GS Mapping 루프 최대 세분화 — 93.0s (12단계)



process_track_data() 전체(camera 생성·add_next_kf·pose 업데이트)까지 계측 범위를 넓혀 12단계로 세분화. rasterize+backward+loss_compute가 여전히 81.4%로 지배적 — 부가작업(camera_init·add_next_kf 등)은 4.2%뿐. map() 내부에 12.0초(12.9%) 미계측이 새로 발견됨(isotropic loss 계산·viewpoint 샘플링 추정, 다음 계측 후보).